Assignment 9 (Capstone)

Financial Blockchain Capstone: An mZAR Remittance Payment Prototype for the South Africa-Zimbabwe Corridor

Blockchain: 262BLCH9X2

Makungu Hlongwane (220029167)

Thapelo Lekgoathi (201578477)

Kgothalang Wesi (220113838)

Part A African remittance payment prototype

Executive Summary

Problem. Sending money from South Africa costs an average of 15.23% of the amount sent, the highest of any G20 country and five times the 3% global target (World Bank, 2025). In the South Africa–Zimbabwe corridor, costs reach 12.7% and transfers are slow (IMF, 2025). Three further issues compound the cost:

- Informal, unrecorded transfers. Migrants who cannot access formal financial services send money through informal couriers instead (IMF, 2025). In one Zimbabwean district informal channels accounted for over 80% of reported transfers, leaving senders with no record, insurance or recourse if money is lost (Nzima, 2017, p. 303).
- Unverifiable stablecoin reserves. A stablecoin is only as safe as the reserves behind it, which users cannot see. When $3.3 billion of USDC's reserves were caught in the Silicon Valley Bank failure in 2023, the coin fell to $0.88 (Howcroft and Jaiswal, 2023). The SARB's 2025 review identifies gaps in the prudential regulation of stablecoin issuers (SARB, 2025, p. 33).
- Costly but essential AML controls. Anti-money-laundering reporting adds to corridor costs (IMF, 2025). Yet South Africa was only removed from the FATF grey list in October 2025, and scrutiny continues (Engineering News, 2025).

We built a prototype payment platform on our semester blockchain codebase. A sender converts simulated rand into mZAR, a stablecoin backed 1:1 by a reserve. The sender transfers it in a signed, mined transaction, and the recipient redeems it for simulated rand with a signed receipt. A payment API and block explorer expose every step.

Results: The demo runs end to end. R1,000 is deposited, 750 mZAR is sent and confirmed after two blocks, and the recipient redeems it for R750. The sender keeps 250 mZAR, fully backed by R250. verify_chain() confirms the full history, and a 300 mZAR overspend is rejected.

Why it matters. Each issue maps to one layer of the prototype:

- Ledger: gives every transfer a record that anyone can check.
- Custody: enforces the 1:1 reserve rule after every event, with each issuance and payout signed by the Reserve agent.
- Verification: links each payout to the transfer that funded it, giving an audit trail.

The chain improves auditability and reserve transparency, but it does not remove FX, cash-out or compliance costs. A public chain is therefore not automatically the right rail (see Financial Design). Real payments, FX conversion and KYC/AML checks are out of scope.

1 Scenario and system architecture

1.1 Scenario and objective

The selected corridor is a worker in South Africa sending value to a family member in Zimbabwe. The deposit is expressed in simulated rand-denominated units. The recipient's payout is also recorded in the same demo units; the Zimbabwe setting identifies the beneficiary's location, not a conversion into Zimbabwean currency. No exchange rate, spread or foreign-currency payout is calculated.

The objective is to complete and verify the full sequence from deposit to payout, while preserving a stated 1:1 reserve rule. The minimum viable product includes sender and recipient wallets, recorded on-ramp conversion, a signed mined transfer, recorded redemption, chain verification and transaction lookup, a payment API, and a human-readable explorer. The implemented demonstration also deliberately rejects invalid payment attempts.

1.2 Components and connections

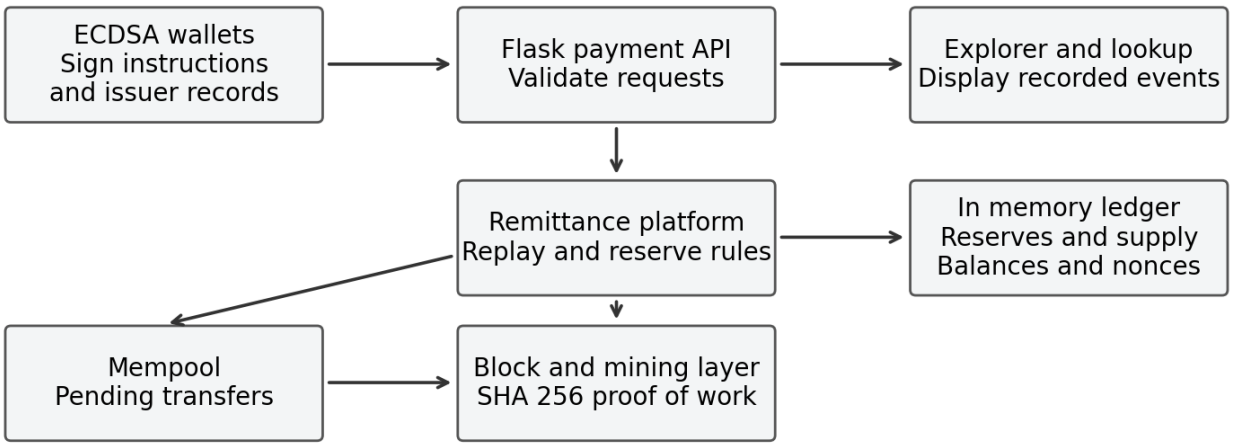


Figure 1. Implemented component relationships. Arrows indicate calls or data flow; all components execute in one process.

Wallets generate signed instructions. Flask routes pass these requests to RemittancePlatform, which applies validation and accounting rules. Valid transfers enter Mempool; mining commits them to Blockchain. The platform maintains the current reserve and account state and can reconstruct it from recorded events. Status, lookup and explorer routes read the same application state.

The notebook's Flask test client exercises the routes without opening a network socket. The optional localhost server exposes the same app to a browser. Consequently, running the explorer does not create a second blockchain or change the payment outcome. This separation makes it possible to demonstrate the entire payment flow inside Jupyter and then show its recorded history in a browser.

2 Data model and implementation decisions

2.1 Accounts reserves and event records

The ledger uses vault_fiat, circulating, balances and nonces. vault_fiat represents simulated reserve units; circulating represents circulating mZAR; balances maps wallet addresses to token holdings; nonces maps sending addresses to the next expected instruction number. Values use integer demo units. There is no fractional token denomination, interest calculation or exchange-rate feed.

A signed instruction includes kind, asset, sender, recipient, amount, timestamp, nonce and public_key. Its SHA-256 transaction identifier is calculated from a canonical representation of the instruction body, and the signature authorises that body. The verifier checks the body, signature, identifier and correspondence between the public key and sender address. SHA-256 addresses record integrity, while ECDSA addresses authorisation; neither check proves a person's legal identity (Antonopoulos, 2015, chs. 4–5).

A dedicated reserve-agent ECDSA wallet signs mint records. Replay checks the signature against the platform's trusted issuer public key, including when an attacker recomputes block hashes. Burn instructions are signed by the redeeming wallet. Blocks store index, timestamp, transactions, previous_hash, nonce and hash. Events and receipts additionally provide readable deposit and payout records. The reserve agent also signs each receipt. It records the burn transaction, block, paid_local and incoming payment identifiers in funding_tx_ids, and labels the payout SIMULATED. The incoming identifiers show recorded payments to the beneficiary; they are not an allocation of individual token units.

2.2 Validation before state changes

The platform validates candidate events against a copy of the ledger before committing a mined block. Transfers require a valid signature, a positive integer amount, the correct sender nonce and sufficient balance. Pending instructions are replayed when checking a new transfer, so available funds already committed to the mempool cannot be promised again. Explicit duplicate checks reject an instruction already pending or mined; nonces additionally enforce instruction order.

Chain verification checks hashes, links, block order and the proof-of-work condition, then replays transactions and compares the reconstructed ledger with the current state. Duplicate transaction identifiers and invalid economic instructions cause verification to fail. This extends basic hash-chain checking with an application-level accounting test.

2.3 Mining and confirmation policy

Difficulty 2 requires a mined hash to begin with 00. Under an ideal uniform hash model, success has probability 1/256 per attempt, or about 256 attempts on average. Genesis has index 0, no transactions, a previous hash of 64 zeros and nonce 0. Its timestamp comes from time.time(), so its hash can differ between runs; identical stored block contents produce the same hash. Genesis is not mined; subsequent blocks must meet the difficulty condition.

Confirmation depth counts the transaction's own block. The payment is included at block index 2, receives one confirmation, then reaches two after an empty block is appended. Redemption is rejected before that threshold. The approach illustrates the confirmation-depth concept without implementing a multi-node fork behaviour. No production confirmation threshold is inferred from the classroom setting (Antonopoulos, 2015, ch. 8).

3 Payment API

3.1 API surface

Method and route	Purpose
POST /onramp	Record a simulated deposit and mint backed mZAR.
POST /transfer or /transactions/new	Validate a signed transfer and add it to the mempool.
POST /mine	Mine a block and return mining statistics.
POST /offramp	Validate a signed redemption, burn tokens and create a receipt.
GET /status	Return balances, vault, supply, nonces and verification flags.
GET /chain and /verify	Return the chain and replay verification results.
GET /issuer	Return the reserve agent address and public key.
GET /receipts/<receipt_id>	Retrieve a signed payout receipt.
GET /transactions/<tx_id>	Locate a recorded transaction and return its confirmation count.
GET /explorer and /explorer/rows	Render readable events or return block summary rows.

The endpoints expose the payment stages through one application. JSON object validation rejects malformed request shapes, and ValueError, KeyError and TypeError are handled as HTTP 400 responses. Unknown transaction or receipt lookup returns HTTP 404; duplicate submission returns HTTP 409. The notebook's call helper asserts the expected status code, so an unexpected acceptance or rejection interrupts the demonstration. A request lock serialises access to the shared in-memory application, it is not a substitute for durable database transactions.

4 Financial design and reserve enforcement

4.1 Stablecoin mechanism

mZAR is a simulated fiat-backed stablecoin with an intended redemption value of one demo rand unit per token. Its stability mechanism is a reserve accounting rule and redemption at the stated rate. The prototype has no secondary market and therefore does not demonstrate a traded price remaining at par. It implements neither an algorithmic supply mechanism nor crypto-collateral liquidation.

Let V denote simulated reserves, S circulating mZAR and B_i the balance of wallet i. At every valid committed state, the prototype requires the following relationship, together with non-negative balances:

$$V = S = \sum_i B_i \geq 0$$

The on-ramp adds the deposit amount to V and S and credits the sender. A transfer debits one wallet and credits another without changing V or S. Redemption debits the recipient, burns the redeemed supply and reduces V by the payout amount. The invariant is checked as part of event application and again in status and replay verification.

Committed stage	Sender	Recipient	Supply	Reserves
Initial state	0	0	0	0
Deposit and mint	1,000	0	1,000	1,000
Transfer mined	250	750	1,000	1,000
Confirmation block	250	750	1,000	1,000
Redeem and burn	250	0	250	250

Table 1. Simulated accounting path. Balances are mZAR; reserves are demo local units. Pending submission does not yet change committed balances.

4.2 Issuance and payout responsibilities

The on-ramp is a trusted local operation: an API request simulates receipt of money and causes the reserve agent to sign a mint. Signature validation prevents unsigned or incorrectly signed mint records from passing replay, but the deposit endpoint has no authenticated caller role or bank confirmation. An internally consistent signed mint can therefore correspond to no real deposit. The simulation verifies issuer authorisation and numerical accounting, not custody of bank assets.

A successful off-ramp records a burn and a simulated receipt together. In production, token destruction and bank payout occur across different systems. A payout failure after burning would require durable workflow state, idempotent settlement references, reconciliation and a recovery or compensation process. The receipt in this prototype confirms a recorded simulation event rather than delivery of cash to a beneficiary.

No mining reward is issued in mZAR, because increasing token supply without adding reserves would breach the stated rule. There are no transaction charges, reserve investment returns, interest payments or operator profits in the model. These omissions simplify accounting and prevent an artificial claim about commercial profitability.

5 Financial risks and suitability of public chains

5.1 Depeg liquidity and custody

A production issuer could maintain a nominal 1:1 balance yet fail to honour redemption if assets lose value, become illiquid or are inaccessible. Depeg risk therefore depends on reserve quality, liquidity, credible redemption and confidence. Lecture 10 distinguishes the code-enforced classroom ratio from a market-supported peg and warns against interpreting the simulation as run-proof.

Practical safeguards would include segregated, high-quality liquid reserves, limits on custodian concentration, independent reconciliation and an enforceable redemption claim. The FSB's recommendations emphasise clear legal claims and timely redemption, including redemption at par into fiat for a single-currency stablecoin (Financial Stability Board, 2023, recommendation 9). This is an international design benchmark, not evidence that mZAR has regulatory approval.

If reserves fall below token liabilities, the prototype has no capital buffer, insurance or legal loss-allocation mechanism. Under a proposed production design, issuer equity would absorb losses first; a shortfall beyond that buffer could expose holders to delayed or reduced recovery, depending on their legal claims and insolvency arrangements. This loss allocation must be established before issuance and cannot be created by hash-chain verification alone.

Custody risk includes loss or misuse of wallet keys and theft, fraud or freezing of fiat assets. Keys are held in notebook memory; reserves are integer counters. Neither has production protection. Real operation would require secure key management, controlled mint permissions, dual authorisation, reliable backups, audit records and separation of client reserves from operating funds. No external price oracle is used, so oracle failure is outside the implemented mechanism; it would become relevant if FX or collateral pricing were added.

5.2 Compliance and corridor economics

Addresses are pseudonymous and do not identify customers. A production on-ramp would need customer identification and sanctions screening, while transfers and redemption would need risk-based monitoring, beneficiary checks and record retention. Any applicable Travel Rule requirements would need assessment by the responsible, regulated service providers. The current code implements none of these controls and collects no real customer identity data.

The corridor's full cost would include FX spreads, funding and agent float, compliance, network charges and cash-out fees. The simulation measures none of them, so no claim of cheaper or faster remittance is justified. Its Zimbabwe payout would require a separately priced conversion and licensed settlement arrangement in a real service.

5.3 When not to use a public chain

For this corridor, a public chain should not be the default if regulated banking or mobile-money partners already provide affordable settlement, consumer recourse and adequate audit access. Sensitive customer data, demand for predictable finality and costly on/off-ramps can outweigh a public record's benefits. A permissioned audit ledger or conventional payment infrastructure may be preferable. Public rails warrant consideration only where shared verification across institutions creates a measurable benefit after total cost, privacy, liquidity and legal settlement requirements are assessed.

6 Results and demo evidence

We present the results by following one payment through its lifecycle, from wallet creation to the recipient receiving the payout. The stages combine the transaction lifecycle (create → sign → validate → mempool → mine → chain) with the end-to-end payment path (on-ramp → signed transaction → mempool → mine → chain → explorer → off-ramp with receipt). The difficulty is specified as 2, and the payout rule has 2 confirmations. The screenshots document the supplied demonstration run.

Stage 1: Wallet creation

We generated three ECDSA key pairs on the SECP256k1 curve using the ecdsa library: one for the Sender (the worker in South Africa), one for the Recipient (the family member in Zimbabwe), and one for the Reserve agent, which issues mZAR and holds the reserve. Following the semester wallet exercise, the keys are fresh throwaway keys generated on every run. Each Wallet keeps its private key internally and never prints it; the private key is used only to sign. The address is the SHA-256 hash of the public key, truncated to its first 40 hexadecimal characters. The public key is derived from the private key, but the private key cannot be derived from the public key, so anyone can verify a wallet's signatures without being able to forge them (Antonopoulos, 2015, ch. 4).

```
Sender address:        1dfb6382c19dc512d7216d04b01d2cb002bc9f19
Recipient address:     2e49af844917f086f41fd458cd68846c8f92cb6f
Reserve agent address: c6d7a7f9edac1f055df7f46ea2a7fbae8613c8bf
```

Figure 2. Wallet creation: the three addresses (private keys are never displayed)

Stage 2: Platform start-up and genesis block

Before any payment, the platform creates the chain with a genesis block. Figure 3 prints every field of the block: index 0, no transactions, a previous hash of 64 zeros, nonce 0, and its timestamp, which is the Unix time at which it was created using time.time(). Reconstructing that stored block preserves its timestamp and produces the same hash; creating a new genesis block at a different time can produce a different hash. check_genesis_previous_hash() confirms that the previous hash is exactly 64 zeros; verify_chain() runs the same check whenever it verifies the chain, and a copy whose previous hash ends in 1 fails it.

```
Blockchain and mempool components ready

=== Genesis ===
  index: 0
  timestamp: 1791729528.780948
  transactions: []
  previous_hash: 0000000000000000000000000000000000000000000000000000000000000000
  nonce: 0
  hash: a9ce93996cf6e66835613e86b85d483c60dd47bb45a6be36818e06161aa97ae5
  previous_hash is 64 zeros: True

Genesis ready
Reserve agent (issuer): c6d7a7f9edac1f055df7f46ea2a7fbae8613c8bf
Blockchain verified: True
verify_chain(): True
```

Figure 3. Genesis block with the previous-hash check

Stage 3: On-ramp: deposit and mint

The Sender deposits R1,000 through POST /onramp, which returns HTTP 201. The platform builds a mint record for 1,000 mZAR, the Reserve agent signs it, and it is mined into block 1. vault_fiat and circulating both rise to 1,000, so the 1:1 reserve rule holds. GET /issuer returns the Reserve agent's public key, and verify_record() uses it to confirm the mint signature ("Mint signed by the Reserve agent: True"). This authenticates issuance by the reserve agent; it does not independently establish receipt or custody of real fiat reserves.

```
POST /onramp → HTTP 201
{
  "amount": 1000,
  "block": 1,
  "circulating_after": 1000,
  "deposited_local": 1000,
  "event": "onramp",
  "issuer_signature": "d5d6e9228c461499c2ac134ddb4eb416b64e87a73cb4ac580b08aa9101c407cbc716bce64
c68891318ed004df970d96405c95fce07db9db43f497a7259337b42",
  "minted_mzar": 1000,
  "tx_id": "e23c550d835202cad74f25932f946c5aecb7e3b8efbc0e1e052683d64c140ea6",
  "vault_fiat_after": 1000
}
GET /issuer → HTTP 200
{
  "address": "c6d7a7f9edac1f055df7f46ea2a7fbae8613c8bf",
  "label": "Reserve agent (issuer)",
  "public_key": "4ea90ea5f5babf6773846f4507360e1a1c6feceb142ec0445a74ef62bf657e0bcf756d7b14a095d
186f4163e20394a5dd64e944ddfa6c13f11adc60adb0eaa8b"
}
Mint signed by the Reserve agent: True
Sender mZAR: 1000
```

Figure 4. On-ramp: R1,000 deposited, 1,000 mZAR minted in block 1 and signed by the Reserve agent

Stages 4–5: Create, sign and queue the payment

The Sender calls Wallet.instruction() to create a 750 mZAR transfer to the Recipient. The instruction carries the five course fields (sender, recipient, amount, timestamp and signature) plus kind, asset, nonce and public_key. The body is serialised as canonical JSON and signed with the Sender's private key; the signature itself is left out of the signed bytes. The tx_id is the SHA-256 hash of the same body. verify_instruction() confirms the signature with the Sender's public key ("Public-key signature verified: True").

The signed instruction is then submitted to POST /transactions/new. Before accepting it, the platform checks that the signature is valid and the address matches the public key, that the nonce is the next one expected, that the transaction is not a duplicate, and that the Sender can afford it after any other pending transfers. It is accepted with HTTP 201 ("Transaction queued") and placed in the mempool for block 2. The lookup shows status pending with 0 confirmations, and the Recipient's balance is still 0: waiting in the mempool is not settlement.

```
Public-key signature verified: True
POST /transactions/new → HTTP 201
{
  "block_index": 2,
  "message": "Transaction queued",
  "pending": 1,
  "status": "pending",
  "tx_id": "f199294a8a035637c0098c988e6230e73521c55bad576b03a0a245f4dde151d2"
}
GET /transactions/f199294a8a035637c0098c988e6230e73521c55bad576b03a0a245f4dde151d2 → HTTP 200
{
  "block": null,
  "confirmations": 0,
  "required_confirmations": 2,
  "status": "pending",
  "transaction": {
    "amount": 750,
    "asset": "mZAR",
    "kind": "transfer",
    "nonce": 0,
    "public_key": "a6435ac826fab0534e47e9db8c6aed007764601dfe78ea6383e487ffa6ff7bfa3b94ade948d0f
079afcc1d5cd6c26332a1cabdf56569467eb2b9b1158b16314a",
    "recipient": "ed87ebc89bdac891198d230a6dc76d363157df76",
    "sender": "22bdda530aeb35b463b0bdbe05b490bd6f5b02d9",
    "signature": "4c0222dcf6209525ffda1d15826e430f4659f66f0f21a7ef1a33d758fa62f8ab596ef07b0ca903
6e8172cdd17c0fc77d0d9cd8917d728a0fdc3c167ccff8f4de",
    "timestamp": 1791729171,
    "tx_id": "f199294a8a035637c0098c988e6230e73521c55bad576b03a0a245f4dde151d2"
  },
  "tx_id": "f199294a8a035637c0098c988e6230e73521c55bad576b03a0a245f4dde151d2"
}
Recipient confirmed balance before mining: 0
```


Figure 5. Signed 750 mZAR transfer accepted into the mempool (status: pending)

Stage 6: Mine the block

POST /mine takes the transactions waiting in the mempool and builds block 2, linked to the hash of block 1. mine_block() then tries nonce values until the block hash starts with two zeros (difficulty 2); here it took 107 attempts and 0.0016 seconds. Before the block joins the chain, append_block() checks its index, its link to the previous block, its hash and its proof of work. The lookup now shows the payment in block 2 with status mined and 1 confirmation, and the Recipient's balance is 750 mZAR.


```
POST /mine → HTTP 200
{
  "hash": "009de3a347fc0340a0509de3dce941c8f3954df3c28c1d95bea9c5592c5944d5",
  "index": 2,
  "message": "New block forged",
  "mining": {
    "attempts": 107,
    "difficulty": 2,
    "nonce": 106,
    "seconds": 0.0016
  },
  "transactions": [
    {
      "amount": 750,
      "asset": "mZAR",
      "kind": "transfer",
      "nonce": 0,
      "public_key": "d6d886c07ebc5d229dc0849124f899e666079678acf0c40c44b13d0b114e4fdafb3d1edb3ab476980c15dd09689118c4fc09b9339c6259c0448e469
028d541ea",
      "recipient": "2e49af844917f086f41fd458cd68846c8f92cb6f",
      "sender": "1dfb6382c19dc512d7216d04b01d2cb002bc9f19",
      "signature": "f84f4017c15a851ebfcb11bafbf6e456a387268f27f60350126257c812f0074cd290001004c570e606e7793e997e250aedab3d0ec9cf2520b58dc27c
44375053",
      "timestamp": 1791730061,
      "tx_id": "f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a"
    }
  ]
}
Proof of work: difficulty 2, 107 attempts, 0.0016 s
GET /transactions/f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a → HTTP 200
{
  "block": 2,
  "block_hash": "009de3a347fc0340a0509de3dce941c8f3954df3c28c1d95bea9c5592c5944d5",
  "confirmations": 1,
  "required_confirmations": 2,
  "status": "mined",
  "transaction": {
    "amount": 750,
    "asset": "mZAR",
    "kind": "transfer",
    "nonce": 0,
    "public_key": "d6d886c07ebc5d229dc0849124f899e666079678acf0c40c44b13d0b114e4fdafb3d1edb3ab476980c15dd09689118c4fc09b9339c6259c0448e46902
8d541ea",
    "recipient": "2e49af844917f086f41fd458cd68846c8f92cb6f",
    "sender": "1dfb6382c19dc512d7216d04b01d2cb002bc9f19",
    "signature": "f84f4017c15a851ebfcb11bafbf6e456a387268f27f60350126257c812f0074cd290001004c570e606e7793e997e250aedab3d0ec9cf2520b58dc27c44
375053",
    "timestamp": 1791730061,
    "tx_id": "f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a"
  },
  "tx_id": "f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a"
```


Figure 6. Block 2 mined by proof of work; the payment now has 1 confirmation

Stage 7: Confirmations

Our payout policy requires 2 confirmations before mZAR can be redeemed, to illustrate a confirmation-gated payout policy. When the Recipient tried to redeem straight away, POST /offramp was refused with HTTP 400 (“Funds have not reached required confirmations”), and the vault stayed at R1,000. We then mined an empty block 3 on top. Confirmations are calculated as k = H − h + 1 (Antonopoulos, 2015, ch. 8): with the tip at height 3 and the payment at height 2, k = 2, so the lookup now shows status confirmed.


```
Attempt redemption too early:
POST /offramp → HTTP 400
{
  "error": "Funds have not reached required confirmations"
}

Mine one confirmation block:
POST /mine → HTTP 200
{
  "hash": "0088170be46ff455b8884991d3d630dc95fc698fd53236ddee292af94926e916",
  "index": 3,
  "message": "New block forged",
  "mining": {
    "attempts": 444,
    "difficulty": 2,
    "nonce": 443,
    "seconds": 0.0022
  },
  "transactions": []
}
GET /transactions/f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a → HTTP 200
{
  "block": 2,
  "block_hash": "009de3a347fc0340a0509de3dce941c8f3954df3c28c1d95bea9c5592c5944d5",
  "confirmations": 2,
  "required_confirmations": 2,
  "status": "confirmed",
  "transaction": {
    "amount": 750,
    "asset": "mZAR",
    "kind": "transfer",
    "nonce": 0,
    "public_key": "d6d886c07ebc5d229dc0849124f899e666079678acf0c40c44b13d0b114e4fdafb3d1edb3ab47
6980c15dd09689118c4fc09b9339c6259c0448e469028d541ea",
    "recipient": "2e49af844917f086f41fd458cd68846c8f92cb6f",
    "sender": "1dfb6382c19dc512d7216d04b01d2cb002bc9f19",
    "signature": "f84f4017c15a851ebfcb11bafbf6e456a387268f27f60350126257c812f0074cd290001004c570
e606e7793e997e250aedab3d0ec9cf2520b58dc27c44375053",
    "timestamp": 1791730061,
    "tx_id": "f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a"
  },
  "tx_id": "f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a"
}
```


Figure 7. Early redemption refused; confirmation block 3 mined; payment confirmed (2/2)

Stage 8: Payment received: redemption and receipt

The Recipient signed a burn instruction for 750 mZAR and submitted it to POST /offramp, which returned HTTP 201. The platform checked the signature and that the funds had reached the required confirmations. It then burned the tokens in block 4, reduced vault_fiat and circulating to 250, and issued a receipt for the simulated R750 payout. The receipt is signed by the Reserve agent, names the burn transaction, and lists the payment’s transaction ID under funding_tx_ids, which links the cash-out to the on-chain transfer that funded it. verify_record() confirms the receipt signature.

```
POST /offramp → HTTP 201
{
  "address": "2e49af844917f086f41fd458cd68846c8f92cb6f",
  "block": 4,
  "block_hash": "0069bf7788761d25682415bbc6e5818f41b26d15c70362b411b4e07350f9f5bf",
  "burn_tx_id": "caa765e6f0840c4942217eeb35d6b38f68577bf9bcb03cf355c0a615baaf4787",
  "burned_mzar": 750,
  "circulating_after": 250,
  "currency": "ZAR demo units",
  "funding_tx_ids": [
    "f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a"
  ],
  "issuer": "c6d7a7f9edac1f055df7f46ea2a7fbae8613c8bf",
  "paid_local": 750,
  "payout": "SIMULATED",
  "receipt_id": "00b9c3a99ebab9c02a9963bb99502a6c1cce8a027da8d80bc9031c041c429b47",
  "signature": "f5155052f858754d1fe25775d8bdf1cd17adc342236f493fd92e601cc5cc96da50d5a7bf37c55fc2
a65c5c83efe86c82fea06b3e6fe296e55083dbaed2ce914b",
  "timestamp": 1791730682.240333,
  "vault_fiat_after": 250
}
GET /receipts/00b9c3a99ebab9c02a9963bb99502a6c1cce8a027da8d80bc9031c041c429b47 → HTTP 200
{
  "address": "2e49af844917f086f41fd458cd68846c8f92cb6f",
  "block": 4,
  "block_hash": "0069bf7788761d25682415bbc6e5818f41b26d15c70362b411b4e07350f9f5bf",
  "burn_tx_id": "caa765e6f0840c4942217eeb35d6b38f68577bf9bcb03cf355c0a615baaf4787",
  "burned_mzar": 750,
  "circulating_after": 250,
  "currency": "ZAR demo units",
  "funding_tx_ids": [
    "f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a"
  ],
  "issuer": "c6d7a7f9edac1f055df7f46ea2a7fbae8613c8bf",
  "paid_local": 750,
  "payout": "SIMULATED",
  "receipt_id": "00b9c3a99ebab9c02a9963bb99502a6c1cce8a027da8d80bc9031c041c429b47",
  "signature": "f5155052f858754d1fe25775d8bdf1cd17adc342236f493fd92e601cc5cc96da50d5a7bf37c55fc2
a65c5c83efe86c82fea06b3e6fe296e55083dbaed2ce914b",
  "timestamp": 1791730682.240333,
  "vault_fiat_after": 250
}
Recipient simulated payout: 750 local units
Receipt signed by the Reserve agent: True
Receipt funded by payment: f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a
```


Figure 8. Payment received: 750 mZAR redeemed for a simulated R750 payout with a signed receipt

Stage 9: Verification and explorer

GET /verify runs verify_chain(), which re-checks every block's hash, link and proof of work, replays every transaction (signatures, nonces and balances) and checks the 1:1 reserve rule. It returned verify_chain: true for the 5-block chain. The payment summary below the verification result shows the lookup for the 750 mZAR transfer: block 2, its hash, 3 confirmations, the sender, the recipient and the amount. Finally, we changed the payment amount from 750 to 751 in a copy of the chain, and verify_chain() rejected it ("Altered-chain test: rejected").


```
GET /verify → HTTP 200
{
  "difficulty": 2,
  "invariant_ok": true,
  "length": 5,
  "verify_chain": true
}

=== Payment summary ===
Transaction ID : f9cca957c40140f3ed182bf7c18e2af48bd06eac6d509165b0d8e872e4050f9a
Block index    : 2
Block hash     : 009de3a347fc0340a0509de3dce941c8f3954df3c28c1d95bea9c5592c5944d5
Confirmations  : 3 (required 2)
From           : Sender 1dfb6382c19dc512d7216d04b01d2cb002bc9f19
To             : Recipient 2e49af844917f086f41fd458cd68846c8f92cb6f
Amount         : 750 mZAR
Status         : confirmed
Receipt ID     : 00b9c3a99ebab9c02a9963bb99502a6c1cce8a027da8d80bc9031c041c429b47
```


Figure 9. verify_chain(), payment summary for the 750 mZAR transfer, and tamper test

The explorer gives a human-readable view of the whole chain, newest block first. For each block it shows the hash, the previous hash, the timestamp, the nonce and the number of confirmations. For each transaction it shows the ID, sender, recipient, amount and type. Every hash after the genesis block starts with 00, meeting difficulty 2, and each block's previous hash matches the hash of the block below it.

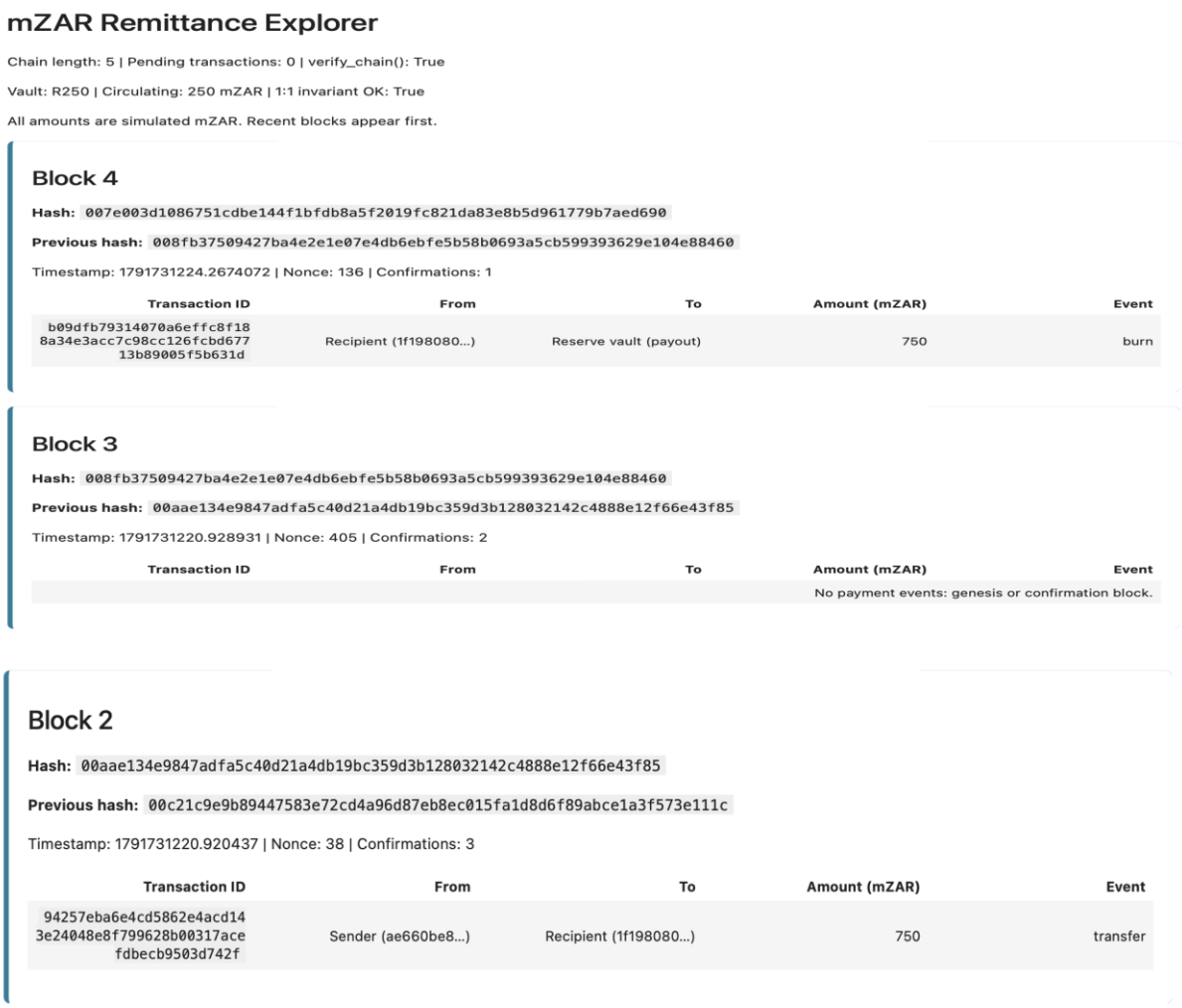


Figure 10. Explorer, part 1: burn (block 4), confirmation block (block 3) and the 750 mZAR transfer (block 2)

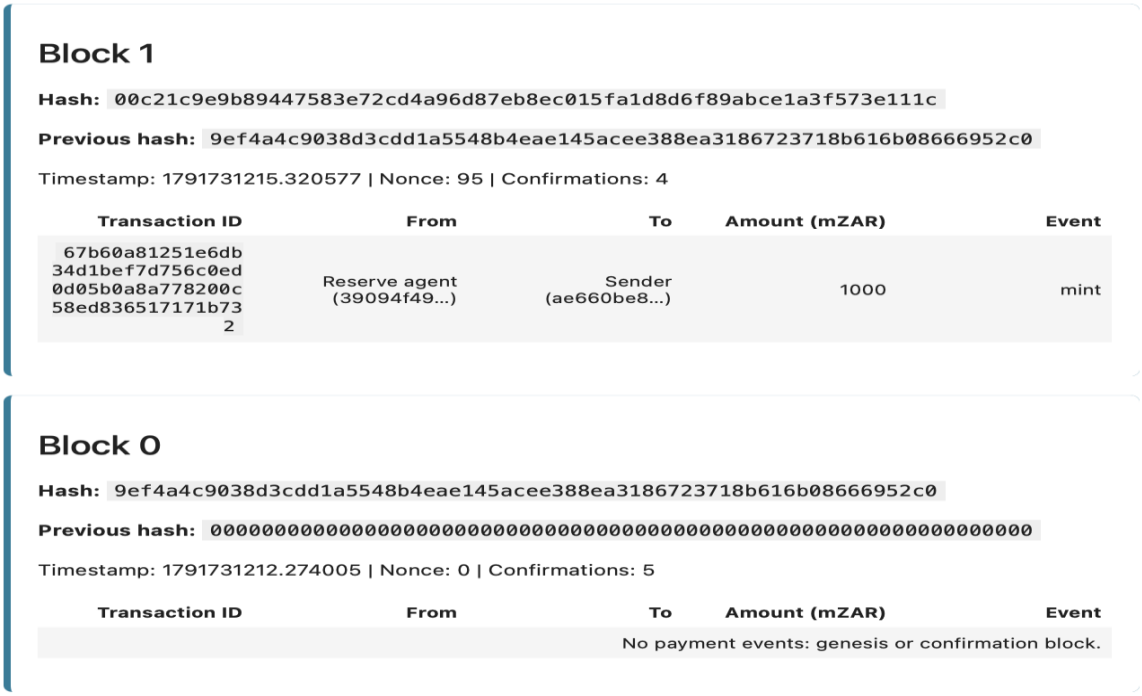


Figure 11. Explorer, part 2: mint (block 1) and genesis (block 0)

Stage 10: Rejected transfers and final balances

GET /status shows the final state: the Sender holds 250 mZAR, the Recipient holds 0, and vault_fiat and circulating are both 250, with the invariant and chain checks both true. We then submitted three invalid transfers, and all three were rejected without changing balances, reserves or the chain (status() was identical afterwards).

```
GET /status → HTTP 200
{
  "balances": {
    "1f198080ffbce5156c26276429c2e75ab8e75e20": 0,
    "ae660be8ed1cc4c98d16de6d943c29c4e3ee120f": 250
  },
  "blocks": 5,
  "chain_ok": true,
  "circulating": 250,
  "invariant_ok": true,
  "nonces": {
    "1f198080ffbce5156c26276429c2e75ab8e75e20": 1,
    "ae660be8ed1cc4c98d16de6d943c29c4e3ee120f": 1
  },
  "pending": 0,
  "vault_fiat": 250
}

Insufficient funds: attempt to send 300 mZAR
POST /transactions/new → HTTP 400
{
  "error": "Insufficient balance"
}

Tampered signature: change a signed amount
POST /transactions/new → HTTP 400
{
  "error": "Invalid signature or transaction ID"
}

Replay: resubmit the mined payment (duplicate → HTTP 409)
POST /transactions/new → HTTP 409
{
  "error": "Duplicate transaction: already pending or on chain"
}

All rejected attempts left balances and reserves unchanged.
```

Figure 12. Final status and three rejected transfers

Table 2. Rejected attempts

Attempt	HTTP	Why it was rejected
Send 300 mZAR with only 250	400	Insufficient balance
Amount changed after signing	400	The signature no longer matches the signed body
Resubmit the mined 750 payment	409	Duplicate: the transaction is already on the chain

Table 3. Balances after each step

Step	Sender mZAR	Recipient mZAR	Vault (R)
Start	0	0	0
On-ramp 1,000	1,000	0	1,000
Transfer 750	250	750	1,000
Off-ramp 750	250	0	250

7 Limitations future work and conclusion

7.1 Scope and limitations

The application has one authoritative process and no peer-to-peer network. Its proof of work links locally produced blocks but offers no network-wide consensus, competing-chain choice or protection against a privileged operator replacing the whole history. Two confirmations demonstrate a rule, not independent economic finality. The confirmation count is relative to the chain currently presented to the verifier (Antonopoulos, 2015, ch. 8).

All reserves and payouts are simulated; the API trusts an on-ramp request as a deposit. No authenticated deposit-caller role, banking integration, legal redemption right, independent reserve attestation or cash-out agent exists. A ledger replay confirms internal consistency but cannot detect whether a custodian has lost external funds. Nor does it recreate a traded stablecoin peg or a market redemption run.

State disappears when the kernel restarts. Keys are demonstration keys in memory, and signed instructions do not include a production chain-domain separation scheme. The API is a localhost teaching service rather than a hardened deployment. There are no persistent records, durable payout workflow, access-control scheme, rate limits, production monitoring or key-recovery process. Token amounts are integer demo units, with no real FX, decimal precision, fees or interest.

Compliance and consumer protection remain conceptual. The prototype cannot substantiate a claim of lower remittance charges or settlement times because it contains no external payment comparison or performance benchmark. Its tests cover selected invariants and failure cases, not all adversarial sequences or operational failures. Receipt funding_tx_ids lists recorded incoming transfers rather than proving exact token provenance across multiple payments.

7.2 Prioritised future work

The first extension should be durable storage and a reconciled payout state machine, with idempotent request identifiers and recovery tests around failed payouts. Second, API access to issuance and redemption should be restricted to authenticated roles, backed by a simulated bank interface that reconciles independently of the token ledger. Third, key handling, audit logging and monitoring should be strengthened, with separation between operational reserves and customer liabilities.

A later multi-node extension could implement fork choice and test reorganisations before determining a value-sensitive confirmation policy. An Ethereum test-network deployment could explore ERC-20 interoperability, but would require new smart-contract code, gas accounting and security review. It would still need reserve and legal controls outside the chain. FX and agent liquidity should be added only with explicit pricing, funding and jurisdictional assumptions.

7.3 Conclusion

The prototype completes the required simulated remittance and preserves its reserve relationship. Its strongest result is the integration of signed instructions, pending-payment controls, mined records, confirmation-gated redemption and verifiable accounting through one API and explorer. The implementation fulfils the educational purpose while showing why trustworthy financial infrastructure also depends on custody, liquidity, compliance and recoverable operations beyond the blockchain.

Part B ERC 20 tokens and the account model

Ethereum's account model and ERC-20 describe different layers of the same system. The account model defines how Ethereum represents accounts and updates their state, while ERC-20 specifies a common interface for fungible tokens implemented through smart contracts. ERC-20 therefore operates within Ethereum's account model rather than replacing it. Ethereum distinguishes externally owned accounts, controlled through private keys, from contract accounts, controlled by executable code (Ethereum.org, 2026a).

Chapter 5 of Mastering Ethereum explains that wallets manage keys rather than physically storing cryptocurrency. Assets are recorded on the blockchain, and wallet software enables users to authorise transactions. This distinction applies to our prototype: the sender and recipient wallets sign payments, and the reserve-agent wallet signs mint records and receipts, while their mZAR balances are maintained in the platform's ledger. However, our demonstration wallets lack hierarchical deterministic key management, mnemonic recovery and encrypted keystores, which the chapter discusses as mechanisms for managing and protecting keys (Ethereumbook, n.d.).

Ethereum records native ETH balances at the protocol level. An ERC-20 token instead maintains its token balances in contract storage, commonly through a mapping from addresses to amounts. An address can therefore hold ETH alongside several distinct token balances. ERC-20 standardises functions including totalSupply, balanceOf and transfer, together with approve, allowance and transferFrom for delegated spending. Its Transfer and Approval events allow applications to observe token activity. These shared interfaces enable compatibility with wallets and other contracts (Vogelsteller and Buterin, 2015).

Our mZAR prototype also uses account balances: a transfer subtracts value from the sender and adds it to the recipient. It does not use Bitcoin's unspent transaction output model. Nevertheless, mZAR is not ERC-20 compliant. Its balances are held in a Python dictionary, while circulating supply and vault_fiat reserves are integer counters, and Flask endpoints invoke application methods rather than Ethereum Virtual Machine code. The prototype has no allowance mechanism or standard ERC-20 event logs. It also rejects zero-value transfers, whereas ERC-20 requires these to be treated as valid transfers that emit an event (Vogelsteller and Buterin, 2015).

Authorisation and settlement also differ. Our prototype signs canonical JSON instructions using ECDSA and enforces application-level nonces and duplicate checks to prevent replay; signed timestamps protect instruction integrity. Its SHA-256-derived addresses are not Ethereum addresses. Ethereum externally owned accounts use Keccak-256-derived addresses and protocol transaction nonces. Ordinary Ethereum transactions incur gas fees paid in ETH, including transactions that call token contracts. Our demonstration charges no fees and uses local proof-of-work mining with two confirmations. Ethereum mainnet currently uses proof of stake, so our confirmation rule does not reproduce its consensus or finality mechanism (Ethereum.org, 2026a–c).

Finally, ERC-20 compliance would not establish stablecoin backing. The standard provides token functionality without guaranteeing fiat reserves or redemption at par. Our prototype enforces equality between simulated reserves, circulating supply and aggregate balances. Issuer signatures authenticate records but do not verify actual bank deposits. An Ethereum implementation could restrict minting and burning through contract permissions, but bank custody, reserve reconciliation and actual payouts would still require external arrangements. ERC-20 deployment would improve interoperability and place token execution on a shared network; it would not independently resolve the custody and financial risks identified in Part A.

Part C Public stablecoins for inbound remittances

Recommendation

Permit a limited, supervised pilot through licensed payment providers before considering wider use of public stablecoin rails. Approval should depend on reliable redemption, effective oversight and evidence of lower total costs for recipients. The objective is to improve inbound remittances while preserving monetary stability and consumer protection. A stablecoin transfer should not be presented as central bank money or proof that the beneficiary has received cash.

Potential benefits and corridor constraints

Public blockchains can provide a shared transaction record and allow transfers outside banking hours. The IMF identifies potential benefits for remittance speed, cost and financial inclusion, while warning that these outcomes depend on sound policy and infrastructure (Adrian, Miccoli and Sugimoto, 2025). These benefits should be tested across the whole payment journey, from the sender's deposit to usable money in the recipient's account or hands.

For the South Africa-Zimbabwe scenario, licensed providers would still need funding, foreign exchange, compliance checks and local payout liquidity. Our mZAR demonstration pays in the same simulated rand units; it does not convert into the recipient economy's currency. Lecture 10 shows why FX spreads, agent liquidity and cash-out costs can remain even when the blockchain transfer is inexpensive. The pilot should compare the recipient's final amount and time to payout against existing bank and mobile-money services.

Risks for the receiving economy

A nominal one-to-one peg can fail when reserves lose value or cannot be accessed quickly. Holders may face a discounted token price or delayed redemption during a run. The FSB calls for clear legal claims, timely redemption and redemption at par into fiat for single-currency stablecoins (Financial Stability Board, 2023, recommendation 9). Public transaction visibility cannot establish reserve ownership, liquidity or recoverability in insolvency.

Foreign-currency stablecoins can encourage households to move savings and payments away from domestic money, weakening monetary policy transmission. They can also reduce authorities' visibility over cross-border flows (Adrian, Miccoli and Sugimoto, 2025). The BIS highlights the difficulty of maintaining the singleness of money when different issuers' tokens trade at different prices, alongside liquidity and financial-integrity concerns (Bank for International Settlements, 2025).

Operational risks include stolen keys, smart-contract faults, congestion, interrupted access and reliance on foreign issuers or service providers. Blockchain inclusion also differs from legal settlement and local cash availability. Pseudonymous addresses require customer identification and monitoring at regulated entry and exit points; an immutable record alone does not identify the sender or beneficiary.

Conditions for a pilot

Require an identifiable, accountable issuer and licensed intermediaries, with cooperation between sending and receiving authorities. Reserve rules should require segregated, high-quality liquid assets, independent reconciliation and transparent reporting. Establish who can redeem, the currency and timing of redemption, and the treatment of holders if the issuer fails. These safeguards follow the FSB's emphasis on governance, risk management, disclosures and cross-border supervision (Financial Stability Board, 2023, recommendations 3-5 and 8-10).

For the proposed pilot, verify customers and beneficiaries, screen sanctions, monitor suspicious activity and meet applicable information-sharing requirements. Protect personal data off-chain. Require secure key custody, incident reporting, complaints handling and a recovery plan for failed payouts. Providers should disclose all fees and FX spreads before acceptance and reconcile the token burn with the actual beneficiary payout.

Use transaction and exposure caps, a defined pilot period and reporting on costs, payout delays, redemption failures, complaints and foreign-currency holdings. Test stressed redemptions and network outages before expansion. Suspend new activity if reserve or payout controls fail, while maintaining an orderly process for existing holders. These are proposed approval conditions, not controls already demonstrated by our code.

Evidence and decision rule

Our prototype records a 1,000-unit deposit, a signed 750 mZAR transfer and a 750-unit redemption, leaving 250 tokens and matching simulated reserves. Issuer-signed records and receipts improve verification, but no real custody, FX or banking settlement occurs. The central bank should therefore treat the demonstration as evidence of payment logic, not readiness for deployment. Expand only if the pilot improves end-to-end outcomes without unacceptable monetary or operational risks. Where established payment systems are cheaper and provide stronger recourse, prioritise those systems.

Reference list

Adrian, T., Miccoli, M. and Sugimoto, N. (2025) How Stablecoins Can Improve Payments and Global Finance. IMF Blog, 4 December. Available at: https://www.imf.org/en/blogs/articles/2025/12/04/how-stablecoins-can-improve-payments-and-global-finance (Accessed: 11 October 2026).

Antonopoulos, A.M. (2015) Mastering Bitcoin: Unlocking Digital Cryptocurrencies. 1st edn. Sebastopol, CA: O'Reilly Media.

Bank for International Settlements (2025) The next-generation monetary and financial system. Annual Economic Report, Chapter III. Available at: https://www.bis.org/publ/arpdf/ar2025e3.htm (Accessed: 11 October 2026).

Engineering News (2025) 'Widespread relief expressed as South Africa removed from FATF grey list', 27 October. Available at: https://www.engineeringnews.co.za/article/widespread-relief-expressed-as-south-africa-removed-from-fatf-grey-list-2025-10-27 (Accessed: 11 October 2026).

Ethereum.org (2026a) Ethereum accounts. Available at: https://ethereum.org/en/developers/docs/accounts/ (Accessed: 11 October 2026).

Ethereum.org (2026b) Ethereum gas and fees. Available at: https://ethereum.org/en/developers/docs/gas/ (Accessed: 11 October 2026).

Ethereum.org (2026c) Proof-of-stake. Available at: https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/ (Accessed: 11 October 2026).

Ethereumbook (n.d.) Mastering Ethereum, Chapter 5: 'Wallets'. Available at: https://github.com/ethereumbook/ethereumbook/blob/develop/src/chapter_5.md (Accessed: 11 October 2026).

Financial Stability Board (2023) High-level Recommendations for the Regulation, Supervision and Oversight of Global Stablecoin Arrangements: Final report. 17 July. Available at: https://www.fsb.org/2023/07/high-level-recommendations-for-the-regulation-supervision-and-oversight-of-global-stablecoin-arrangements-final-report/ (Accessed: 11 October 2026).

Howcroft, E. and Jaiswal, R. (2023) 'Stablecoin USDC breaks dollar peg after revealing $3.3 bln Silicon Valley Bank exposure', Reuters, 11 March (via Kitco News). Available at: https://www.kitco.com/news/off-the-wire/2023-03-11/stablecoin-usdc-breaks-dollar-peg-after-revealing-33-bln-silicon (Accessed: 11 October 2026).

International Monetary Fund (IMF) (2025) South Africa: Technical Assistance Report – Cross-Border Payments: South Africa–Zimbabwe Corridor Diagnostic Report. Technical Assistance Report No. 2025/093, 5 November. Washington, DC: International Monetary Fund. Available at: https://www.imf.org/en/publications/technical-assistance-reports/issues/2025/11/05/south-africa-technical-assistance-report-cross-border-payments-south-africa-zimbabwe-571586 (Accessed: 11 October 2026).

Nzima, D. (2017) 'Channelling migrant remittances from South Africa to Zimbabwe: opportunities and obstacles', Alternation, 24(1), pp. 294–313. Available at: https://doi.org/10.29086/2519-5476/2017/v24n1a14 (Accessed: 11 October 2026).

South African Reserve Bank (SARB) (2025) Financial Stability Review. Second edition, November. Pretoria: South African Reserve Bank. Available at: https://www.resbank.co.za/en/home/publications/publication-detail-pages/reviews/finstab-review/2025/second-edition (Accessed: 11 October 2026).

van Flymen, D. (2020) Learn Blockchain by Building One: A Concise Path to Understanding Cryptocurrencies. Berkeley, CA: Apress. https://doi.org/10.1007/978-1-4842-5171-3.

Vogelsteller, F. and Buterin, V. (2015) 'ERC-20: Token Standard', Ethereum Improvement Proposals, 20. Available at: https://eips.ethereum.org/EIPS/eip-20 (Accessed: 11 October 2026).

World Bank (2025) Remittance Prices Worldwide, Issue 53, March 2025. Washington, DC: World Bank. Available at: https://remittanceprices.worldbank.org/sites/default/files/rpw_main_report_and_annex_q125_1_0.pdf (Accessed: 11 October 2026).

Appendix A Attribution contributions and requirement coverage

A.1 Attribution and AI declaration

The group's semester work supplies the hashing, wallet, mempool, block, mining and API foundations. Lecture 10 supplies the stablecoin payment and reserve-accounting teaching framework. The capstone adapts those components into the mZAR deposit, transfer and redemption flow. Confirmation depth is adapted conceptually from A7; distributed consensus and A5 Merkle proofs are not implemented in this version.

OpenAI ChatGPT and Anthropic Claude assisted with code adaptation, integration, review, debugging and testing, as declared in the submitted notebook. OpenAI ChatGPT also assisted with report drafting and interpretation of outputs. Group members are responsible for reviewing the sources, understanding and validating the implementation, and explaining it during the live demonstration. The report does not claim independently audited software or real financial transactions.

A.2 Individual contributions

Makungu Hlongwane contributed to coding, integration and debugging. She also coordinated the supporting materials and report preparation, checked the references and consolidated the final document.

Kgothalang Wesi contributed to coding, integration and debugging. Kgothalang also developed the financial design and policy discussion, covering reserve backing, redemption, remittance risks and the recommendations in Part C.

Thapelo Lekgoathi contributed to coding, integration and debugging. Thapelo also checked the demonstration outputs and rejection tests, organised the screenshots and results explanations.

A.3 Minimum feature coverage

The table above maps the prototype's minimum required features to the corresponding report sections and demonstration evidence.

Requirement	Implemented evidence
Wallet creation	Sender, recipient and reserve-agent ECDSA wallets; Sections 2 and 6, Figure 2.
Simulated on-ramp	1,000-unit deposit and mint record; Sections 4 and 6, Figure 4.
Signed mined transfer	750 mZAR in block 2; Section 6, Figures 5–6.
Simulated off-ramp	750 mZAR burn and issuer-signed payout receipt; Sections 4 and 6, Figure 8.
Verification and lookup	Chain replay, transaction lookup and tamper rejection; Sections 2 and 6, Figure 9.
Payment API	Deposit, transfer, mine, redemption and read routes; Section 3.
Human-readable explorer	Rendered HTML block and transaction view; Section 6, Figures 10–11.
Rejected invalid transfer	Overspend and tampering return HTTP 400; replay returns HTTP 409; Section 6, Figure 12 and Table 2.